

Software Design for a Modular Semi-Humanoid Robot

A high-level and low-level architecture for cognition, autonomy, safety and fleet operations

Technical design report | August 2026

Abstract

This report defines an implementation-oriented software architecture for a wheeled semi-humanoid robot with a mobile base, vertical lift, dual arms, tactile hands and an expressive perception head. It synthesizes the supplied literature reviews, market analyses and cross-project architecture into a multi-rate system that separates non-deterministic cognitive inference from deterministic motion execution and independently enforced safety. The high-level stack comprises speech and perception, Vision-Language-Action (VLA) proposals, mission orchestration, navigation, manipulation, digital-twin validation and fleet operations. The low-level stack comprises hardware abstraction, fieldbus gateways, native RTOS control tasks, impedance control, tactile/contact processing, power-aware diagnostics and fail-safe state transitions. The recommended design assigns explicit ownership to each rate domain, limits AI outputs to validated skills and constraints, and treats telemetry, container release management and simulation-to-hardware tests as first-class parts of the product.

Keywords: ROS 2; Nav2; MoveIt 2; `ros2_control`; micro-ROS; RTOS; EtherCAT; CAN-FD; VLA; BehaviorTree.CPP; digital twin; functional safety.

1 Purpose, scope and design method

1.1 System boundary

The target system is a semi-humanoid mobile manipulator: an omnidirectional/Mecanum or swerve mobile base, one-two DoF lift/spine, two 6-7 DoF arms, modular soft-rigid hands, a 3-DoF perception head, RGB-D/LiDAR/IMU sensors, tactile arrays and an edge GPU plus microcontrollers. It is designed for indoor manipulation, facility interaction, laboratory/logistics tasks and research teleoperation. It is not a claim of autonomous operation in every unstructured environment.

1.2 Source-driven methodology

The design reconciles four supplied evidence streams: (1) arXiv and OpenAlex literature reviews, including dedicated software reviews; (2) market reports identifying software, services, OTA and fleet operations as the recurring-value layer; (3) the systems project synthesis defining the candidate hardware, power and project features; and (4) technology benchmarks in the market dossier. Reported numerical rates and product claims are treated as design anchors to validate, not guaranteed performance.

1.3 Design principles

1. **Safety independence:** AI, planning and middleware cannot directly bypass local actuator limits or emergency-stop logic.
2. **Rate separation:** each task runs at the slowest rate consistent with its physical role; hard loops are not scheduled by non-real-time callbacks.
3. **Skill mediation:** language/VLA outputs resolve to typed, constrained and observable robot skills rather than raw motor current.
4. **Modular contracts:** hardware, nodes, fieldbuses and data products expose versioned interfaces that allow component replacement.
5. **Simulation before exposure:** policies and releases progress through model-in-the-loop, SIL, vHIL and limited on-robot validation.

Table 1: Software-driving requirements derived from the supplied sources.

Requirement	Design response	Acceptance evidence
Natural-language and visual task input	Speech-to-text, semantic perception and a VLA/VLM planner produce typed goals	Goal schema validity; no direct actuator command from AI
Mobile bimanual work	Nav2 plus MoveIt 2/whole-body coordination; lift and base included in planning model	Collision-free plan, footprint and reach checks
Contact-rich grasping	Wrist/tactile sensing, slip/contact state estimator and guarded motion	Force tracking, slip recovery and stop tests
Human co-presence	Independent limits, speed zoning, E-stop and MCU safety watchdog	Fault injection and contact-response traces
Serviceability	ROS 2 lifecycle nodes, hardware IDs, diagnostics and configuration profiles	Module discovery, replacement and calibration test
Fleet deployment	Container images, signed OTA, telemetry, WMS/IoT integration and rollback	Release promotion and rollback drill
Sim-to-real	URDF/OpenUSD digital twin, synthetic data and domain randomisation	SIL/vHIL and physical correlation report

Evidence basis: Supplied literature/software reviews, market software analysis, and projects synthesis architecture.

2 Requirements and operating assumptions

3 Reference architecture

3.1 Multi-rate execution model

The central architectural decision is an explicit multi-rate pipeline. Cognitive models may operate at approximately 5–10Hz in the supplied literature, mission and planning services at tens of hertz, middleware and device bridges around 100–500Hz, and motor/contact safety loops at 1kHz. These frequencies must be budgeted and measured per hardware configuration.

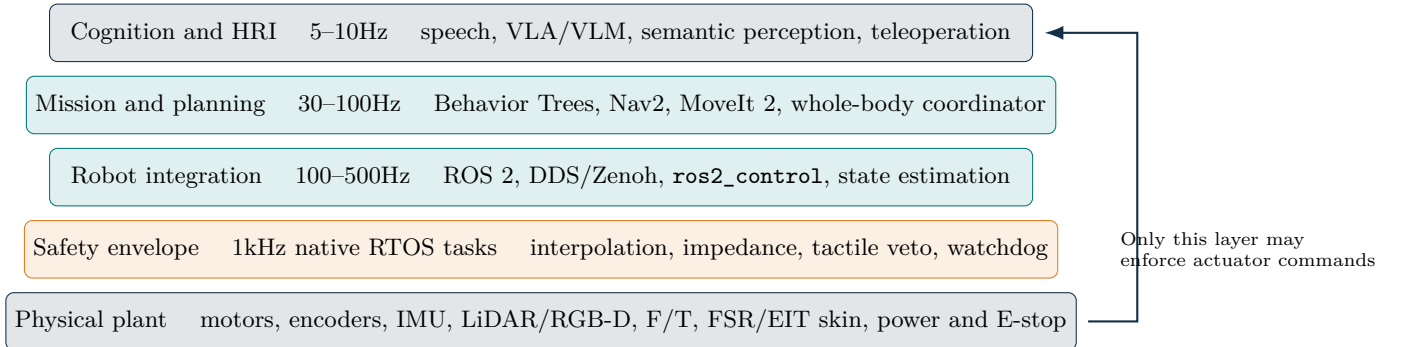


Figure 1: Reference multi-rate software architecture. High-level components propose intent; the safety envelope owns physical execution.

3.2 Command contract

Every layer consumes and emits a constrained artefact: a VLA produces a proposed goal, the mission manager selects a skill, the planner produces a time-stamped trajectory, the interpolator produces bounded set-points, and the MCU produces actuator commands after safety checks. Let $\Delta \mathbf{x}_{AI}$ be an AI-proposed Cartesian delta. It is admissible only after projection into the currently verified feasible set:

$$\mathbf{x}_{\text{ref}} = \Pi_{\mathcal{F}(\mathbf{q}, \varepsilon, S)}(\mathbf{x} + \Delta \mathbf{x}_{AI}), \quad (1)$$

where $\mathcal{F}$ encodes joint, collision, workspace, base-stability, speed-zone and safety constraints; $\mathcal{E}$ is the current environment model; and $\mathcal{S}$ is the safety state. A rejected projection is an observable recovery event, not a silent failure.

4 High-level software design

4.1 Cognition, perception and interaction

The high-level compute domain runs on Jetson Orin AGX/Nano or an industrial GPU host. It ingests stereo/RGB-D, LiDAR, IMU, microphone and facility data. Proposed components are: Whisper-class speech-to-text; VLA/VLM models such as OpenVLA or Pi0; YOLOv11 object detection; SAM/instance segmentation; pose/scene services such as FoundationPose and nvblox/TSDF/ESDF; and a head-attention service using TDOA sound direction, gaze target and expression display. Open-vocabulary 3D grounding, panoramic perception and world-action models are candidate research modules, not safety-critical dependencies.

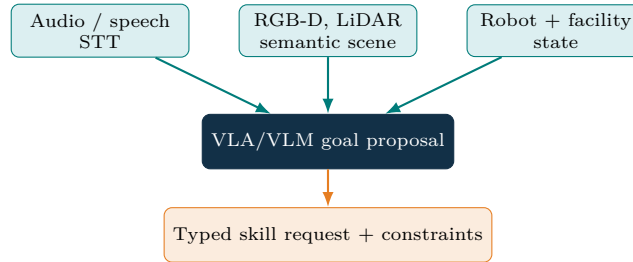


Figure 2: Cognitive data flow. The only output to autonomy is a typed, constrained skill request.

4.2 Mission orchestration and recoverable skills

Use BehaviorTree.CPP as the mission-control surface, with explicit lifecycle and recovery nodes. A top-level tree should sequence **AcquireGoal**, **ValidateContext**, **Navigate**, **Observe**, **Manipulate**, **Verify**, **Deliver** and **Report**. Each action has timeout, cancellation, degraded-mode and recovery semantics. Required bimanual recovery nodes include object slip, grasp failure, base obstruction, unreachable pose, perception loss, communication loss, door/elevator failure and operator handover.

Table 2: Representative high-level skills and their required guards.

Skill	Planner / service composition	Preconditions and verification
Navigate to workstation	Nav2 SLAM, costmaps, MPPI/DWB, speed zones	Localisation valid; footprint free; arrival tolerance met
Pick / place	Perception, MoveIt 2, grasp planner, tactile close	Object confidence; collision-free plan; grasp/contact verification
Bimanual carry / handover	Whole-body coordinator, dual-arm plans, base motion	Object constraint; synchronisation; human/obstacle clearance
Door / elevator routine	Guarded force motion, perception, building/WMS interface	Access granted; force limit; fallback/operator request
Charge / dock	Nav2 docking, BMS and charging-contact service	Battery threshold; charger detection; current/thermal checks

4.3 Navigation, manipulation and whole-body coordination

Nav2 owns localisation, global route generation, local obstacle avoidance and speed-zone compliance. MoveIt 2 owns arm kinematics, collision checking and trajectory generation. `ros2_control` owns controller lifecycle and command transport. For mobile manipulation, the base and lift must appear in the robot model, even when coarse base positioning is scheduled separately. A whole-body coordinator resolves the common stop-then-move failure mode by choosing whether base, lift and arms may move concurrently; it may use model-predictive or SDF-based optimisation where validated.[1, 2]

4.4 Middleware and QoS

ROS 2 Jazzy/Humble is the integration backbone. DDS remains a suitable default for discovery and QoS; `rmw_zenoh` is a candidate transport where shared-memory local IPC or constrained wireless links justify it. QoS policy is data-class specific: high-rate images/point clouds use best-effort, bounded queues and freshness constraints; joint state, action results, safety state and configuration use reliable delivery with explicit deadlines/liveliness. The safety loop must not rely on DDS delivery to stop a robot.

5 Low-level software design

5.1 Hardware abstraction and fieldbus topology

The low-level architecture is divided between a Linux-side `ros2_control` hardware interface and native MCU firmware. EtherCAT is recommended for high-axis-count, synchronised servo configurations; CAN-FD/CANopen is an appropriate cost-sensitive distributed-control alternative. The project synthesis identifies VESC/ODrive BLDC controllers, dual STM32H7/ESP32 microcontrollers and dirty 24–48V versus isolated clean 12/19V domains. This maps to a hardware inventory service that exposes serial number, firmware revision, calibration, fault state and power-domain diagnostics for every module.

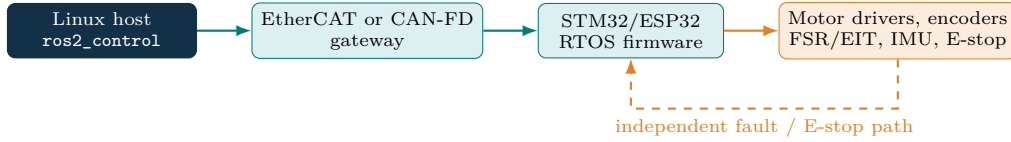


Figure 3: Control-path ownership. ROS 2 requests motion; native firmware enforces timing and physical limits.

5.2 Native real-time firmware

The MCU executes FreeRTOS, Zephyr or a carefully bounded bare-metal schedule. The 1kHz motor/impedance task runs natively and must be decoupled from micro-ROS callback handling; micro-ROS is an asynchronous telemetry, parameter and command bridge. A representative scheduling design is: (i) highest priority: hardware E-stop, current/overtemperature and deadline monitor; (ii) 1kHz: encoder acquisition, FOC/servo update, impedance/tactile filter and actuator limit; (iii) lower priority: fieldbus RX/TX; (iv) lowest priority: micro-ROS, logs and calibration requests.

5.3 Trajectory interpolation and contact control

High-level deltas at 5–10Hz cannot drive motors directly. A trajectory service converts accepted goals into bounded, time-stamped reference segments; a cubic spline or jerk-limited generator produces $q_d(t)$ and $\dot{q}_d(t)$. The local controller calculates

$$\tau_{\text{cmd}} = K_p(q_d - q) + K_d(\dot{q}_d - \dot{q}) + \tau_{\text{ff}}, \quad \mathbf{F}_{\text{cmd}} = \mathbf{K}(\mathbf{x}_d - \mathbf{x}) + \mathbf{D}(\dot{\mathbf{x}}_d - \dot{\mathbf{x}}). \quad (2)$$

The MCU clamps torque, velocity and power; when tactile, F/T or EIT contact is above a configured envelope, it reduces stiffness, freezes/retreats according to the skill policy, and reports a safety event. The supplied sources cite a 25N threshold for one tactile-veto proposal; the deployed threshold must be risk-assessed for the specific hand, task and standard.

5.4 Safety state machine

6 Digital twin, data and deployment operations

6.1 Simulation and sim-to-real

Isaac Sim/Isaac Lab with OpenUSD is recommended for sensor twins, synthetic data, scalable policy training and SIL/vHIL workflows. MuJoCo provides a complementary environment for contact-rich and tendon-driven studies; Gazebo remains a useful open Nav2/MoveIt integration baseline. The promotion sequence is

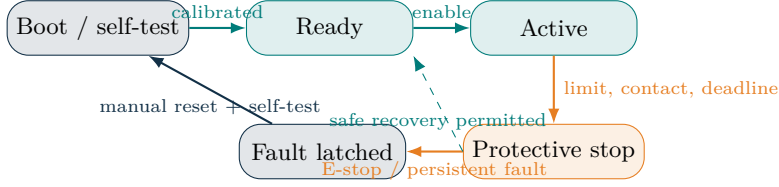


Figure 4: Safety states. The exact transition conditions require a hazard analysis and hardware implementation; no software-only state is accepted as an E-stop substitute.

Model-in-the-Loop $\rightarrow$ SIL $\rightarrow$ vHIL $\rightarrow$ supervised hardware $\rightarrow$ bounded autonomous deployment. Domain randomisation covers lighting, textures, sensor noise/latency, mass, friction, wheel slip and selected contact parameters.[3, 4, 5, 6]

6.2 Data products and teleoperation

VR/web teleoperation provides demonstrations and operator assistance. Every episode should synchronise sensor data, robot state, command state, safety events, calibration versions and environment/task labels. Store raw data separately from curated training datasets; record consent, retention and access controls where people appear in audio/video. Dataset quality should be measured by task coverage, intervention count, distribution shift and replay success—not only episode count.

6.3 Containerised releases, OTA and fleet services

The market analysis identifies software/services, OTA and fleet operations as a major value layer. Package high-level applications in tested OCI/Docker images, with immutable release manifests covering ROS distribution, CUDA/TensorRT, model version, map/skill set and configuration schema. The safety firmware has a separately controlled, signed update path. Fleet updates are staged: simulation, bench hardware, canary robot, limited fleet and broad fleet; every phase has health gates and rollback. WMS/ERP/IoT integration should be asynchronous and unable to command motion directly.

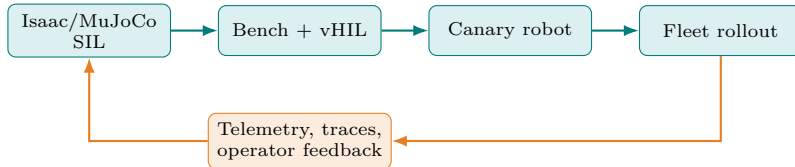


Figure 5: Release and learning lifecycle. Health gates, observability and rollback are part of the runtime design.

7 Interfaces, packages and verification plan

7.1 Recommended package boundaries

Table 3: Package-level ownership and interfaces.

Package / domain	Responsibilities	Contract / verification
robot_description	URDF/Xacro, SRDF, TF frames, collision geometry, limits	Versioned model; CI validates transforms, joint limits and collision pairs
perception_stack	Sensor drivers, calibration, detection, segmentation, pose and scene map	Timestamp/frame convention, confidence and freshness fields

Package / domain	Responsibilities	Contract / verification
<code>mission_bt</code>	Typed goal acceptance, skill sequencing, recovery and operator escalation	Deterministic tree tests, timeout/cancellation coverage
<code>navigation_stack</code>	SLAM, localisation, Nav2, docking, costmaps and speed zones	Map/version identity, localisation covariance, safety-zone API
<code>manipulation_stack</code>	MoveIt 2, grasping, whole-body coordination, guarded motion	Time-stamped trajectory, constraints and verification event
<code>control_bridge</code>	<code>ros2_control</code> hardware interface, state/command conversion	Controller lifecycle; command saturation and stale-command tests
<code>mcu_firmware</code>	RTOS tasks, servo/FOC, tactile safety, watchdog, fieldbus	WCET, jitter, fault and E-stop test traces
<code>fleet_ops</code>	OTA, telemetry, device inventory, release gates, WMS adapters	Signed manifests, audit record, rollback and access-control tests

7.2 Verification matrix

Test stage	What is tested	Exit criterion
Unit / static analysis	Schema, kinematics, command limits, controller logic, firmware memory safety	All safety-relevant paths reviewed and tested
Simulation / SIL	Planning, perception, task trees, domain randomisation, failure injection	Defined success and no unhandled safety transitions
Bench / vHIL	Fieldbus, timing, actuator emulator, tactile inputs, power transients	Measured timing budget and safe response to injected faults
Supervised hardware	Calibrated robot, guarded motion, E-stop, recovery	Safety-state traces and task-level acceptance metrics
Pilot / fleet	OTA, WMS integration, observability, uptime and intervention rate	Canary health gate, rollback demonstrated

8 Implementation roadmap

Phase 1—Safe platform baseline. Establish robot model, hardware inventory, sensor drivers, `ros2_control`, native 1kHz firmware, E-stop, diagnostics and simulation. No VLA is required.

Phase 2—Bounded autonomy. Add Nav2, MoveIt 2, behavior-tree missions, guarded grasps, docking and teleoperation. Validate recovery and hardware-in-the-loop tests.

Phase 3—Cognitive assistance. Introduce speech, semantic perception and VLA-proposed skill parameters behind typed goal validation and operator supervision.

Phase 4—Fleet productisation. Add container release management, signed OTA, WMS/IoT connectors, telemetry dashboards and data governance. Measure deployment effort, availability and intervention rate.

9 Conclusion

The recommended software design makes autonomy useful without making it the only line of defence. ROS 2, Nav2, MoveIt 2 and `ros2_control` supply a modular high- and mid-level foundation; RTOS-native firmware, fieldbus discipline and independent safety state transitions own physical execution; Isaac/OpenUSD and teleoperation support iteration; and fleet operations turn the robot into a maintainable product. This is the appropriate architecture for a semi-humanoid system whose novelty lies in dependable, modular integration across cognition, manipulation and safe real-world operation.

References

- [1] A. Bonci, F. Gaudeni, and M. Giannini, “ROS2-Based Frameworks for Increasing Robot Autonomy: A Survey,” *Applied Sciences*, 2023. DOI: <https://doi.org/10.3390/app132312796>.
- [2] Y. Li, L. Yin, and T. Zhang, “Real-time Whole-Body Motion Planning for Mobile Manipulators Carrying Arbitrarily Shaped Payloads via Kinematically-Coupled SVSDF,” arXiv:2608.07005, 2026. <https://arxiv.org/abs/2608.07005>.
- [3] NVIDIA, “Isaac AI Robot Development Platform,” <https://developer.nvidia.com/isaac>.
- [4] M. Mittal et al., “Isaac Lab: High-Performance Robot Learning Framework Built on NVIDIA Isaac Sim,” arXiv:2403.07257, 2024. <https://arxiv.org/abs/2403.07257>.
- [5] DeepMind, “MuJoCo,” <https://mujoco.org/>.
- [6] Open Robotics, “Gazebo,” <https://gazebo.org/>.
- [7] M. Iovino et al., “A Survey of Behavior Trees in Robotics and AI,” *Robotics and Autonomous Systems*, 2022. DOI: <https://doi.org/10.1016/j.robot.2022.104096>.
- [8] M. J. Kim et al., “OpenVLA: An Open-Source Vision-Language-Action Model,” arXiv:2406.09246, 2024. <https://arxiv.org/abs/2406.09246>.
- [9] N. Stoufs et al., “Budget-Based Real-Time Executor for Micro-ROS,” arXiv:2105.05590, 2021. <https://arxiv.org/abs/2105.05590>.